MarvelBase

A NoSQL Web Application for the Marvel Cinematic Universe

MASTER ADEO2 — NoSQL Project

Author:	Harsh Prajapati & Charmi Shah
Date:	April, 2026
Stack:	Node.js • Express • MongoDB Atlas
Dataset:	Marvel Cinematic Universe Films

CY Cergy Paris Université — 2025/2026

Contents

1 Introduction **2**

2 Architecture **2**
2.1 MVC Pattern 2
2.2 Project Structure 2

3 Data Model **3**
3.1 Movie Schema 3
3.2 Dataset 3

4 Database Connection **4**

5 Features & Functionality **4**
5.1 Main Collection Page 4
5.2 Add, Edit and Delete 5
5.3 Rating System 6
5.4 Statistics Dashboard 6
5.5 Dark / Light Mode 8

6 REST API **8**

7 Technologies Used **9**

8 How to Run **9**

9 Conclusion **9**

1. Introduction

This project was developed as part of the MASTER ADEO2 NoSQL course. The goal was to build a fully functional web application backed by a NoSQL database, allowing users to browse, add, edit, delete, and rate a dataset of their choice.

I chose the **Marvel Cinematic Universe (MCU)** as the dataset because it provides rich, real-world data: box office figures, budgets, directors, cast members, ratings, and release ordering across five distinct phases. The application is called **MarvelBase**.

The full source code is structured according to the **MVC pattern** as required, using Node.js with Express for the backend, EJS for server-side templating, and MongoDB Atlas for cloud-hosted NoSQL storage via Mongoose ODM.

2. Architecture

2.1. MVC Pattern

The application strictly follows the Model-View-Controller pattern:

- **Model** — `models/Movie.js`: Mongoose schema defining the data structure and virtual fields.
- **View** — `views/*.ejs`: EJS templates rendered server-side (index, stats, 404).
- **Controller** — `controllers/movieController.js`: All business logic, CRUD operations, rating system and statistics aggregation.
- **Routes** — `routes/movieRoutes.js`: Maps HTTP requests to controller methods.
- **Database** — `server.js`: Mongoose connection to MongoDB Atlas on startup.

2.2. Project Structure

```
marvel-nosql/
          server.js                    # Entry point, DB connection
          models/
                Movie.js                 # Mongoose schema (Model)
          controllers/
                movieController.js       # Business logic (Controller)
          routes/
                movieRoutes.js           # URL routing
          views/
                index.ejs                # Main collection page (View)
                stats.ejs                # Statistics dashboard (View)
                404.ejs
                partials/
                    header.ejs
                    footer.ejs
          public/
                css/style.css
                js/
                    main.js
                    theme.js
          seed.js                      # Database seeder
          .env                         # Environment variables
```

Listing 1: Project directory structure

3. Data Model

3.1. Movie Schema

The core entity is a `Movie` document stored in MongoDB. Below is the Mongoose schema definition:

```
const movieSchema = new mongoose.Schema({
  title:       { type: String, required: true },
  year:        { type: Number, required: true },
  phase:       { type: Number, required: true, min: 1, max: 5 },
  mcuOrder:    { type: Number, required: true },
  director:    { type: String, required: true },
  runtime:     { type: Number },           // minutes
  budget:      { type: Number },           // million USD
  boxOffice:   { type: Number },           // million USD
  cast:        { type: [String] },
  description: { type: String },
  posterUrl:   { type: String },
  ratings:     { type: [Number], default: [] },
  views:       { type: Number, default: 0 },
}, { timestamps: true });
```

Listing 2: Movie Mongoose schema (models/Movie.js)

Two **virtual fields** are computed on-the-fly and not stored in the database:

- `averageRating` — the mean of the `ratings` array, rounded to one decimal place.
- `roi` — return on investment: $\frac{boxOffice - budget}{budget} \times 100$

3.2. Dataset

The database was seeded with **36 real MCU films** spanning Phase 1 to Phase 5. All data is authentic:

Table 1: Sample of seeded movies with real score data

#	Title	Year	Box Office	IMDb	RT Aud.
6	The Avengers	2012	$1,519M	8.0	91%
19	Avengers: Infinity War	2018	$2,048M	8.4	91%
22	Avengers: Endgame	2019	$2,799M	8.4	90%
27	Spider-Man: No Way Home	2021	$1,901M	8.2	99%
34	Deadpool & Wolverine	2024	$1,338M	7.7	94%

Initial ratings were seeded using a conversion of real **IMDb** scores and **Rotten Tomatoes Audience** scores to a 0–5 scale:

$$target = \frac{(IMDb2) + (RT\%20)}{2}$$

Vote counts are proportional to each film's worldwide box office gross (Endgame receives 200 votes, The Incredible Hulk receives 46).

4. Database Connection

The application connects to **MongoDB Atlas** (cloud-hosted) using the Mongoose ODM. The connection string is stored securely in a `.env` file and never committed to version control.

```
mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    console.log('Connected to MongoDB Atlas');
    app.listen(PORT);
  })
  .catch(err => {
    console.error('DB connection error:', err.message);
    process.exit(1);
  });
```

Listing 3: MongoDB Atlas connection (server.js)

The `MONGODB_URI` follows the standard Atlas format:

```
mongodb+srv://<user>:<password>@<cluster>.mongodb.net/marveldb
```

5. Features & Functionality

5.1. Main Collection Page

The home page displays all MCU films sorted by MCU release order. A live search bar and phase filter (Phase 1–5) let users narrow the list in real-time without any page reload.

Selecting a film populates a detail panel showing the movie poster, title, director, synopsis, star rating, and eight spec cards: year, runtime, box office, budget, ROI, page views, MCU order and total votes. The full cast is displayed as a row of tags below.

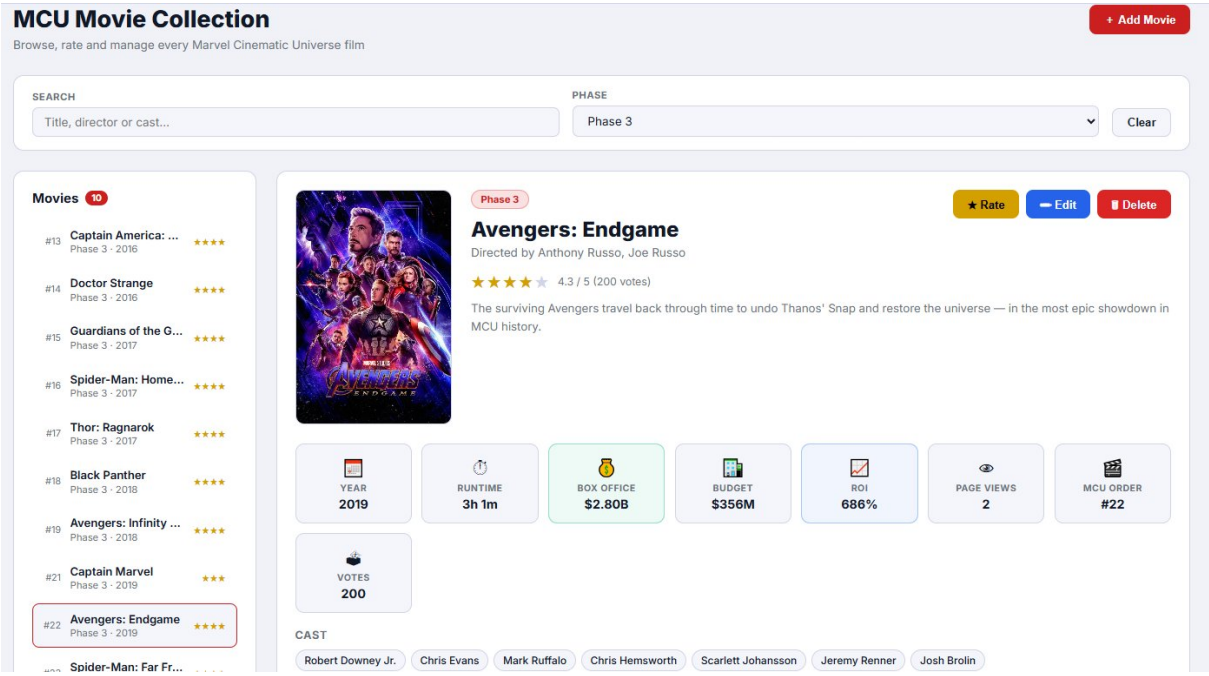

Figure 1: Main collection page — movie list (left) and detail panel for Avengers: Endgame (right), filtered by Phase 3

5.2. Add, Edit and Delete

Users can add a new movie by clicking the *Add Movie* button in the top-right corner. A modal form collects all fields: title, year, phase, MCU order, runtime, director, budget, box office, cast, poster URL and description. The same modal is reused for editing.

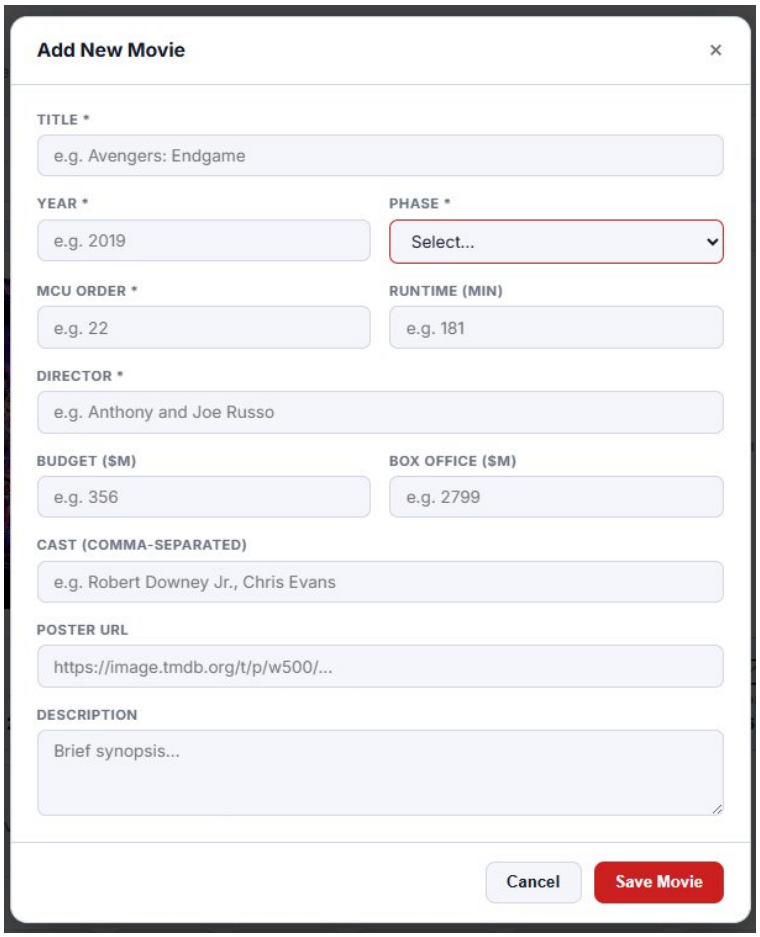

Figure 2: Add New Movie modal with all input fields

Deleting a movie triggers a confirmation dialog before the document is removed from MongoDB. All three operations refresh the list instantly without reloading the page.

5.3. Rating System

Each movie can be rated by clicking the *Rate* button in the detail panel header. A star picker modal opens and the user clicks one of five stars. The vote is pushed into the `ratings` array in MongoDB using `$push`, and the displayed average updates immediately.

5.4. Statistics Dashboard

A dedicated statistics page gives an analytical overview of the entire collection. The page is divided into three areas.

KPI cards at the top summarise the collection at a glance:

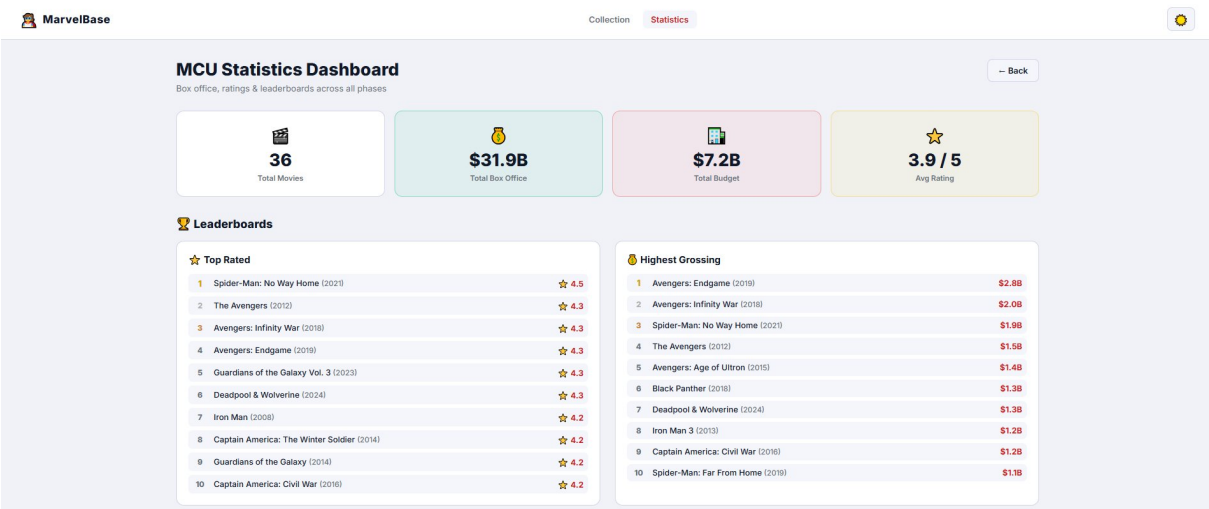

Figure 3: Statistics dashboard — KPI cards ($31.9B total box office, 3.9/5 avg rating) and leaderboards

Leaderboards rank the top 10 movies across four categories: Top Rated, Highest Grossing, Most Viewed and Best ROI. Spider-Man: No Way Home leads the ratings with 4.5/5, while Avengers: Endgame tops the box office at $2.8B.

Charts (powered by Chart.js) visualise the data in five ways:

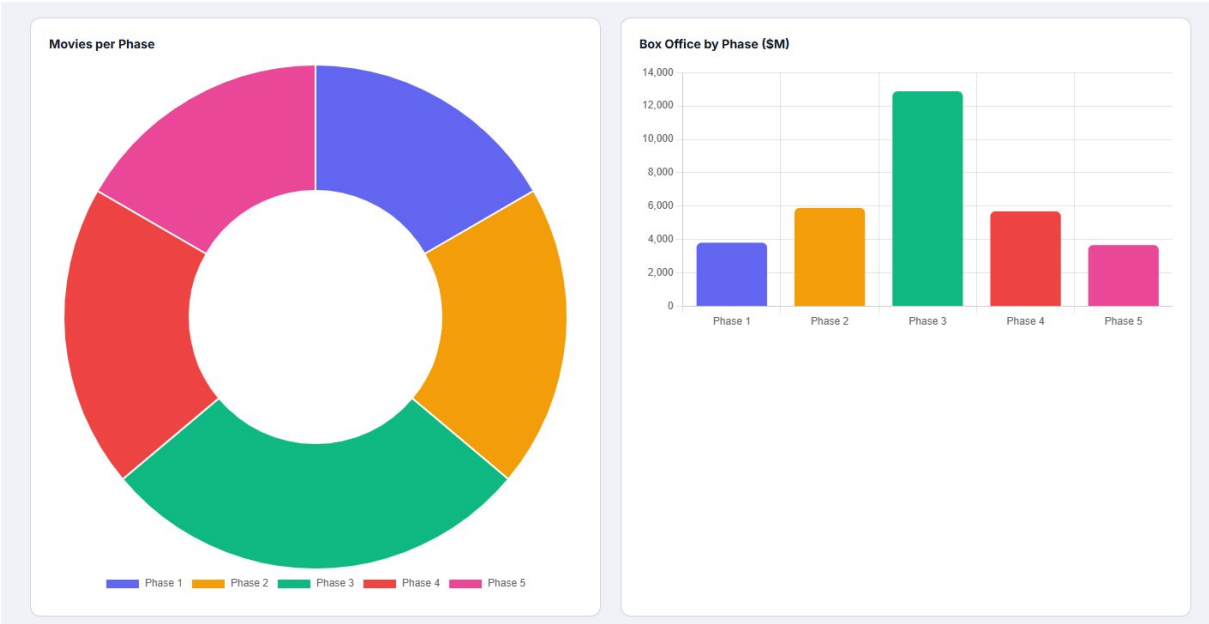

Figure 4: Movies per Phase (doughnut chart) and Box Office by Phase bar chart — Phase 3 dominates with over $12B

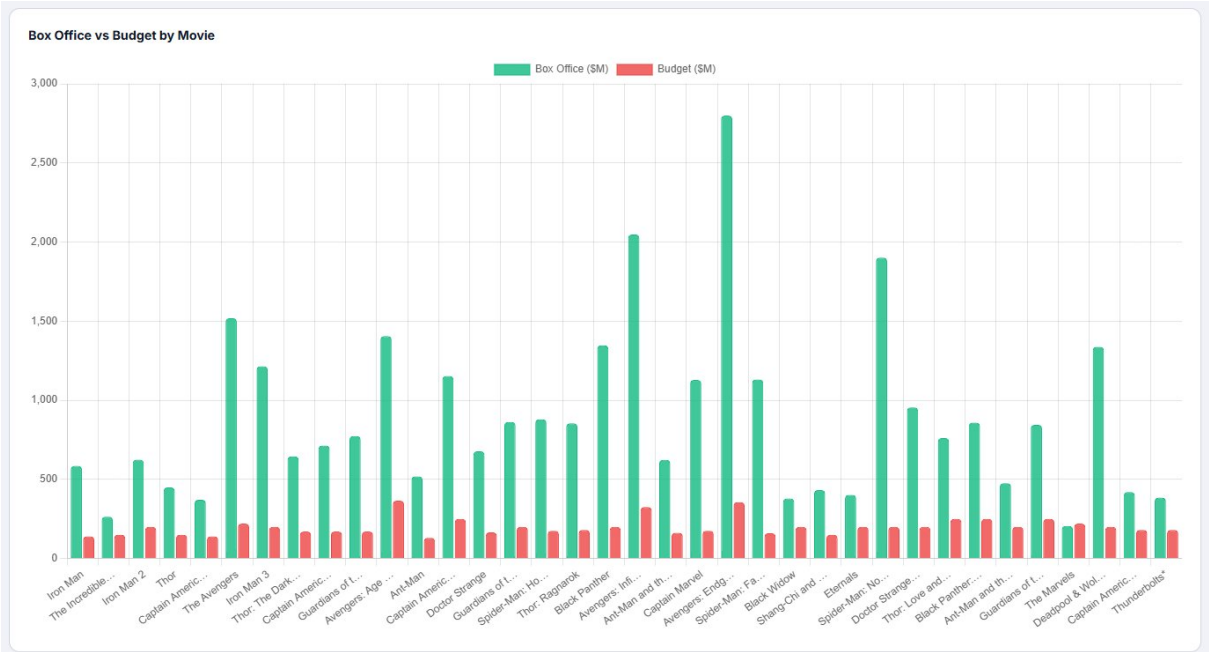

Figure 5: Box Office vs Budget grouped bar chart across all 36 MCU films — Endgame's $2.8B gross is clearly visible

5.5. Dark / Light Mode

The interface supports a dark and light theme, toggled via the sun/moon button in the navbar. The preference is persisted in `localStorage` so it survives page refreshes and navigation between pages.

6. REST API

The backend exposes a clean REST API consumed by the frontend JavaScript. All data is exchanged as JSON.

Table 2: API endpoints

Method	Endpoint	Description
GET	`/api/movies`	List all (supports `?search=&phase=`)
GET	`/api/movies/:id`	Get one movie (increments view count)
POST	`/api/movies`	Create a new movie
PUT	`/api/movies/:id`	Update movie details
POST	`/api/movies/:id/rate`	Submit a rating (0–5)
DELETE	`/api/movies/:id`	Delete a movie
GET	`/api/stats`	Full statistics and leaderboard data

7. Technologies Used

Table 3: Technology stack

Layer	Technology
Runtime	Node.js v18+
Web framework	Express.js 4
NoSQL database	MongoDB Atlas (cloud)
ODM	Mongoose 8
Template engine	EJS
Frontend	Vanilla JavaScript, CSS custom properties
Charts	Chart.js 4
Environment config	dotenv

8. How to Run

1. Unzip the project folder and open a terminal inside it.
2. Install dependencies:

```
npm install
```

3. Copy `.env.example` to `.env` and fill in your MongoDB Atlas URI.
4. Seed the database with all 36 MCU films:

```
node seed.js
```

5. Start the server:

```
npm start
```

6. Open `http://localhost:3000` in your browser.

9. Conclusion

MarvelBase demonstrates the practical use of a document-oriented NoSQL database in a real web application. MongoDB Atlas proved well-suited to this dataset: the flexible schema accommodated nested arrays (`ratings`, `cast`), virtual computed fields avoided redundant storage, and the cloud deployment made the configuration straightforward.

The MVC architecture kept the code clean and easy to maintain. The statistics dashboard and leaderboards show how aggregation over a NoSQL collection can produce meaningful analytics without the need for complex joins.

Overall the project fulfilled all the requirements of the assignment: browse, add, edit, delete and rate a dataset, with a working cloud database connection, a strict MVC structure, and a statistics page for administration.